Introduction to pin muxing



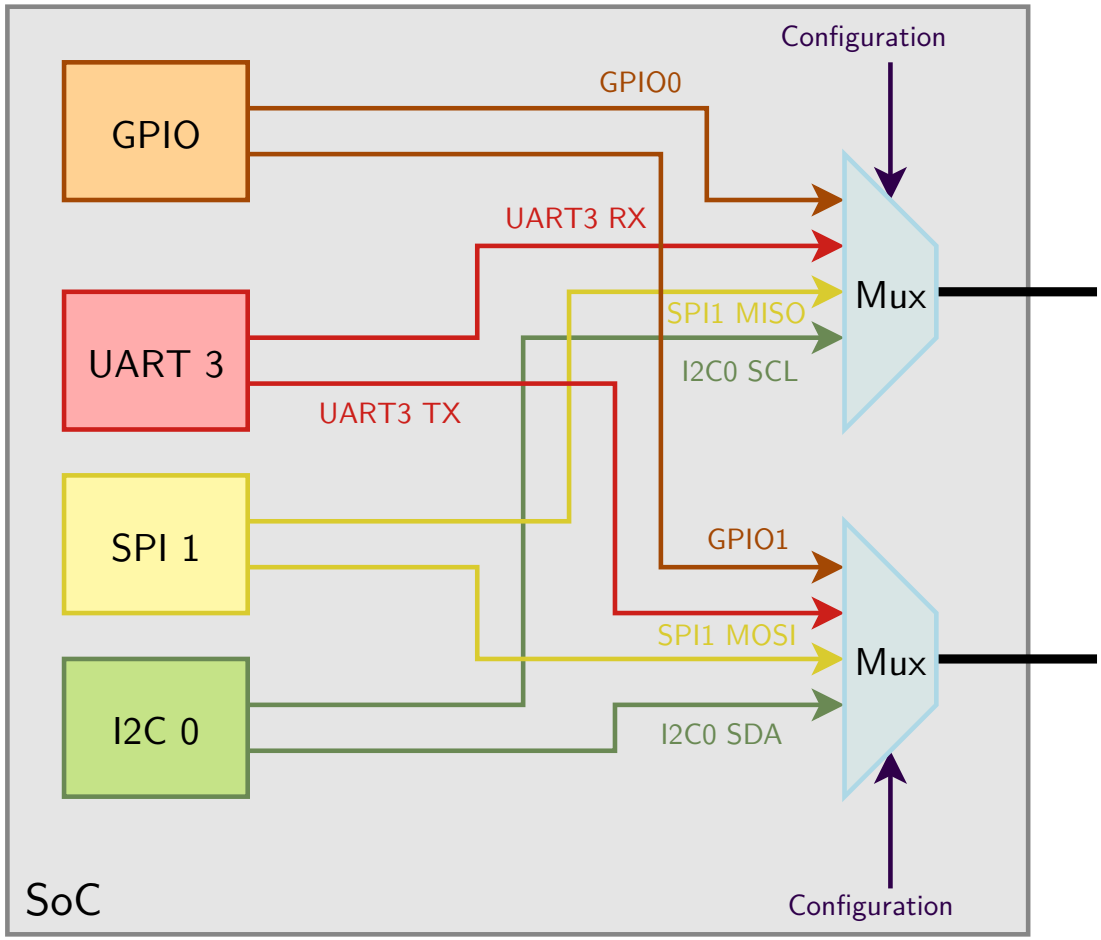
© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



What is pin muxing?

- Modern SoCs (System on Chip) include more and more hardware blocks, many of which need to interface with the outside world using *pins*.
- However, the physical size of the chips remains small, and therefore the number of available pins is limited.
- For this reason, not all of the internal hardware block features can be exposed on the pins simultaneously.
- The pins are **multiplexed**: they expose either the functionality of hardware block A **or** the functionality of hardware block B.
- This *multiplexing* is usually software configurable.

Pin muxing diagram



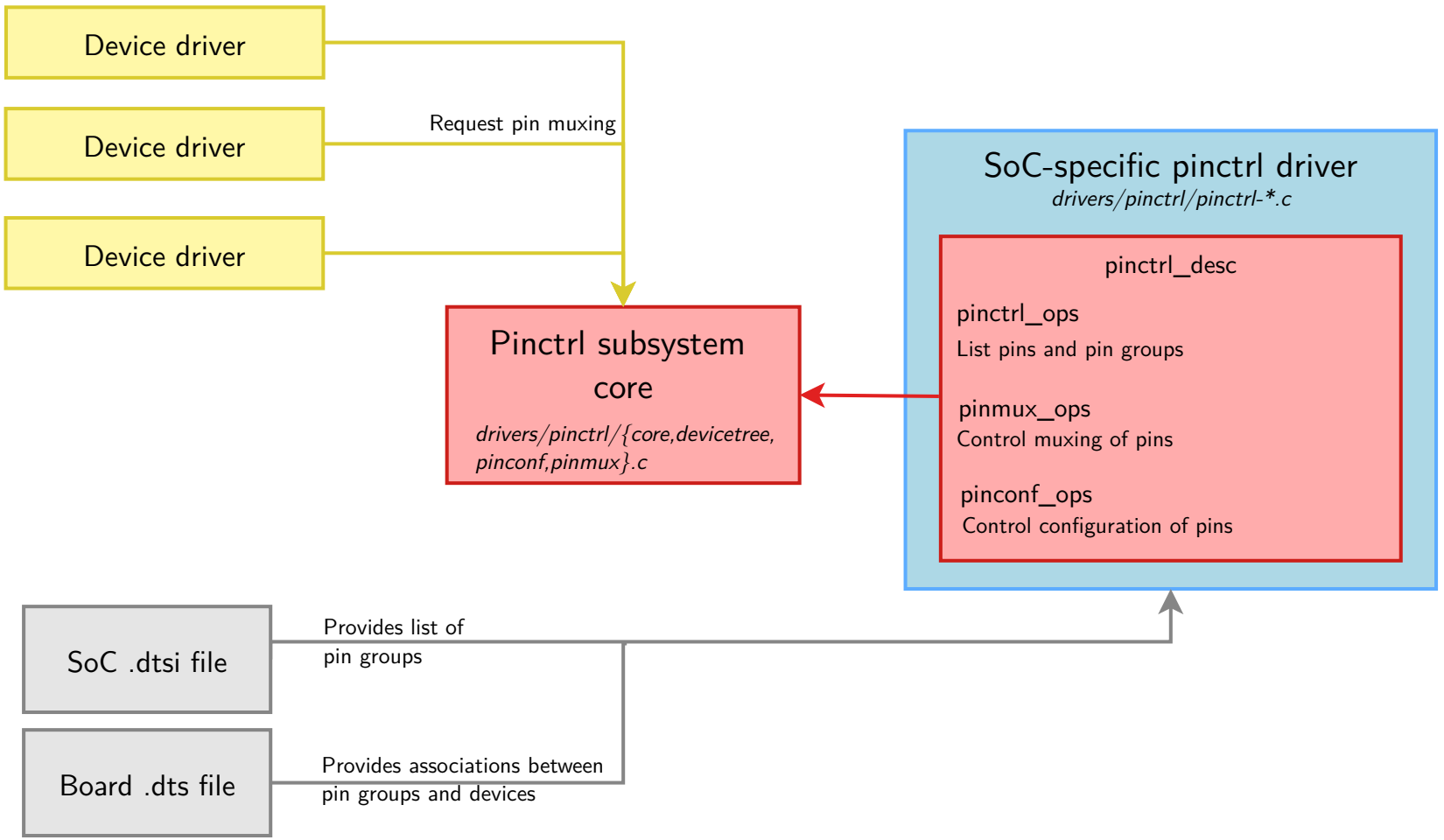


Pin muxing in the Linux kernel

- Since Linux 3.2, a `pinctrl` subsystem has been added.
- This subsystem, located in drivers/pinctrl/ provides a generic subsystem to handle pin muxing. It offers:
  - A pin muxing driver interface, to implement the system-on-chip specific drivers that configure the muxing.
  - A pin muxing consumer interface, for device drivers.
- Most *pinctrl* drivers provide a Device Tree binding, and the pin muxing must be described in the Device Tree.
  - The exact Device Tree binding depends on each driver. Each binding is defined in Documentation/devicetree/bindings/pinctrl.



pinctrl subsystem diagram





Device Tree properties for consumer devices

The devices that require certains pins to be muxed will use the `pinctrl-<x>` and `pinctrl-names` Device Tree properties.

- The `pinctrl-0`, `pinctrl-1`, `pinctrl-<x>` properties link to a pin configuration for a given state of the device.
- The `pinctrl-names` property associates a name to each state. The name `default` is special, and is automatically selected by a device driver, without having to make an explicit *pinctrl* function call.
- See Documentation/devicetree/bindings/pinctrl/pinctrl-bindings.txt for details.



Device Tree properties for consumer devices - Examples

```
i2c0: i2c@11000 {
        ...
        pinctrl-0 = <&pmx_twsi0>;
        pinctrl-names = "default";
        ...
};
```

Most common case (arch/arm/boot/dts/marvell/kirkwood.dtsi)

```
i2c0: i2c@f8014000 {
        ...
        pinctrl-names = "default", "gpio";
        pinctrl-0 = <&pinctrl_i2c0>;
        pinctrl-1 = <&pinctrl_i2c0_gpio>;
        ...
};
```

Case with multiple pin states (arch/arm/boot/dts/microchip/sama5d4.dtsi)



Defining pinctrl configurations

- The different *pinctrl configurations* must be defined as child nodes of the main *pinctrl device* (which controls the muxing of pins).
- The configurations may be defined at:
  - the SoC level (`.dtsi` file), for pin configurations that are often shared between multiple boards
  - at the board level (`.dts` file) for configurations that are board specific.
- The `pinctrl-<x>` property of the consumer device points to the pin configuration it needs through a DT *phandle*.
- The description of the configurations is specific to each *pinctrl driver*. See Documentation/devicetree/bindings/pinctrl for the pinctrl bindings.



Example on OMAP/AM33xx

- On OMAP/AM33xx, the `pinctrl-single` driver is used. It is common between multiple SoCs and simply allows to configure pins by writing a value to a register.
  - In each pin configuration, a `pinctrl-single,pins` value gives a list of *(register, value)* pairs needed to configure the pins.
- To know the correct values, one must use the SoC and board datasheets.

```
/* Excerpt from am335x-bone-common.dts */

&am33xx_pinmux {
   ...
   i2c2_pins: pinmux_i2c2_pins {
      pinctrl-single,pins = <
         /* uart1_ctsn.i2c2_sda */
         AM33XX_PADCONF(AM335X_PIN_UART1_CTSN,
                        PIN_INPUT_PULLUP, MUX_MODE3)
         /* uart1_rtsn.i2c2_scl */
         AM33XX_PADCONF(AM335X_PIN_UART1_RTSN,
                        PIN_INPUT_PULLUP, MUX_MODE3)
      >;
   };
};

&i2c2 {
   pinctrl-names = "default";
   pinctrl-0 = <&i2c2_pins>;

   status = "okay";
   clock-frequency = <400000>;
   ...

   pressure@76 {
      compatible = "bosch,bmp280";
      reg = <0x76>;
   };
};
```



Example on the Allwinner A20 SoC

SoC level

```
/ {
    ...
    soc {
        ...
        pio: pinctrl@01c20800 {
            compatible = "allwinner,sun7i-a20-pinctrl";
            reg = <0x01c20800 0x400>;
            ...

            uart0_pb_pins: uart0-pb-pins {
                pins = "PB22", "PB23";
                function = "uart0";
            };
            ...
        };
    };
};
```

UART0 pin mux config

arch/arm/boot/dts/sun7i-a20.dtsi

Board level

```
/ {
...
    leds {
        compatible = "gpio-leds";
        pinctrl-names = "default";
        pinctrl-0 = <&led_pins_olinuxino>;

        led {
            label = "a20-olinuxino-micro:green:usr";
            gpios = <&pio 7 2 GPIO_ACTIVE_HIGH>;
            default-state = "on";
        };
    };
...
};


&pio {
    ....

    led_pins_olinuxino: led_pins@0 {
        pins = "PH2";
        function = "gpio_out";
        drive-strength = <20>;
    };

...
};

&uart0 {
    pinctrl-names = "default";
    pinctrl-0 = <&uart0_pb_pins>;
    status = "okay";
};
```

Declare LED device and associate pin mux config

LED pin mux config

Enable UART0 and associate pin mux config

arch/arm/boot/dts/sun7i-a20-olinuxino-micro.dts

Practical lab - Setup pinmuxing to enable I2C communication



- Configure the pinmuxing for the I2C bus used to communicate with the Nunchuk
- Validate that the I2C communication works with user space tools.